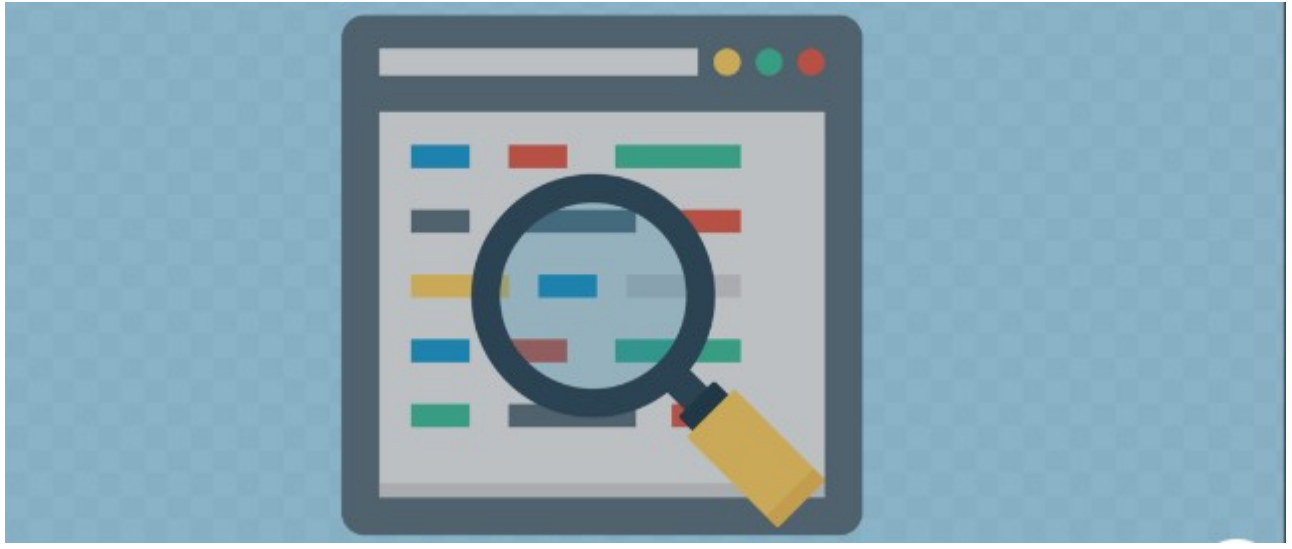
Unidad 2:

Procesamiento de Texto
Pesado de Términos



Vectores para representar el texto

Tras obtener la secuencia ordenada de tokens la podermos utilizar para los distintos modelos. Pero estos no trabajan con la secuencia tal cual, sino que usan una representación mas compacta de un documento

Se asocia a cada token un peso que refleje la importancia del término en el documento

$$D \rightarrow \{(t_1, w_1), (t_2, w_2) \ldots\}$$

	...	amor	atentados	carrera	cuenta	de	espías	fuego	judio	la	munich	novela	olimpiadas	película	que	relato	...
A	...	0	1	0	1	5	1	0	0	2	2	2	1	1	5	2	...

4. Ponderación de términos

- Asignación de un peso a cada término que nos indique la importancia del mismo en el documento.



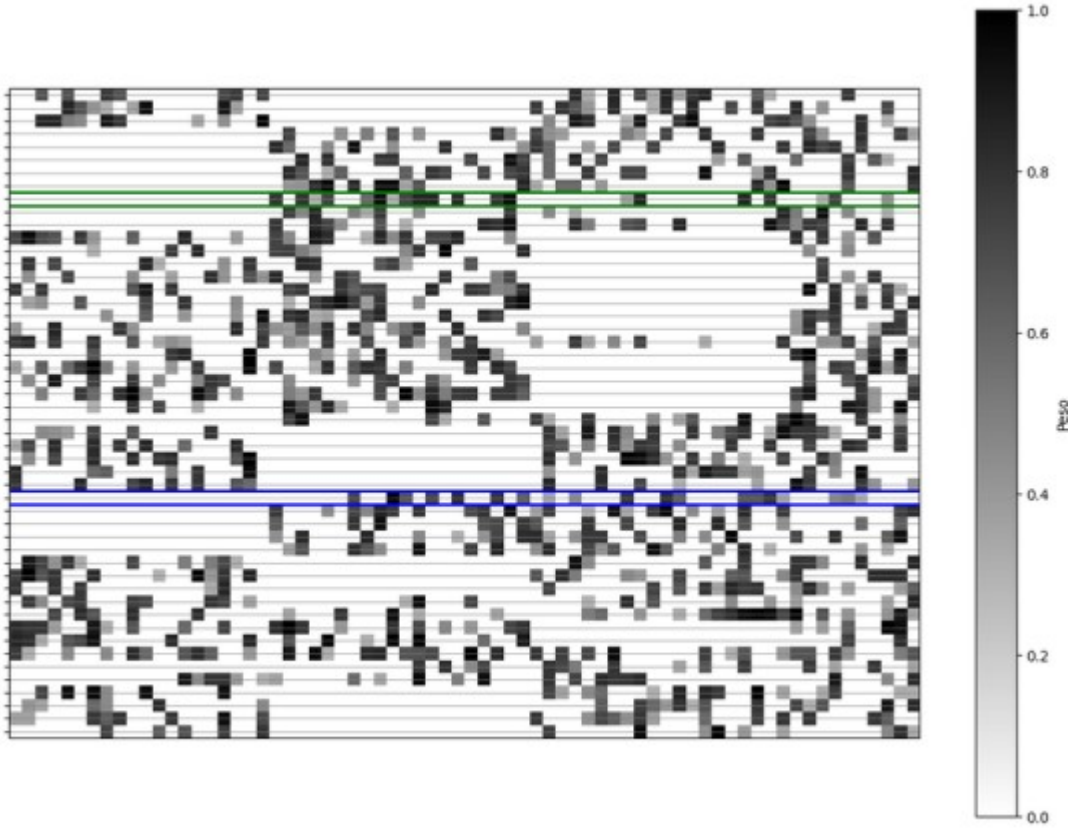
4. Ponderación de términos

- Estos pesos nos permiten:

  - Discriminar entre documentos relevantes y no relevantes.

  - Calcular similitud entre documentos

- Serán utilizados por los modelos de recuperación para obtener la salida.
- Muchas variantes, dependiendo del modelo de recuperación utilizado



4. Ponderación de términos

- Esquema de ponderación binario:
  - Considera la presencia (1) o ausencia de un término en los documentos de la colección.
-

docs	*t1*	*t2*	*t3*
D1	1	0	1
D2	1	0	0
D3	0	1	1
D4	1	0	0
D5	1	1	1
D6	1	1	0
D7	0	1	0
D8	0	1	0
D9	0	0	1
D10	0	1	1



Sólo tenemos información de que un término aparece en un documento.

4. Ponderación de términos

- Esquema basado en la frecuencia de aparición del término i-ésimo (term frequency – tf_{ij}) en el documento j-ésimo.

docs	*t1*	*t2*	*t3*
D1	2	0	3
D2	1	0	0
D3	0	4	7
D4	3	0	0
D5	1	6	3
D6	3	5	0
D7	0	8	0
D8	0	10	0
D9	0	0	1
D10	0	3	5

- Un término que aparezca más veces en un documento será probablemente más importante que si lo hace sólo una vez.
- Aunque... los términos aparecerán más en documentos más largos sin ser necesariamente mas relevantes.
- Normalización de la frecuencia por la máxima frecuencia en el documento o por la longitud del mismo.

Variantes

Variants of TF weight

weighting scheme	TF weight
binary	$0, 1$
raw frequency	$f_{t,d}$
log normalization	$1 + \log(f_{t,d})$
double normalization 0.5	$0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$
double normalization K	$K + (1 - K) \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$

Fuente: Wikipedia

4. Ponderación de términos

- Frecuencia inversa del término i-ésimo en la colección (inverse document frequency – idf_i).

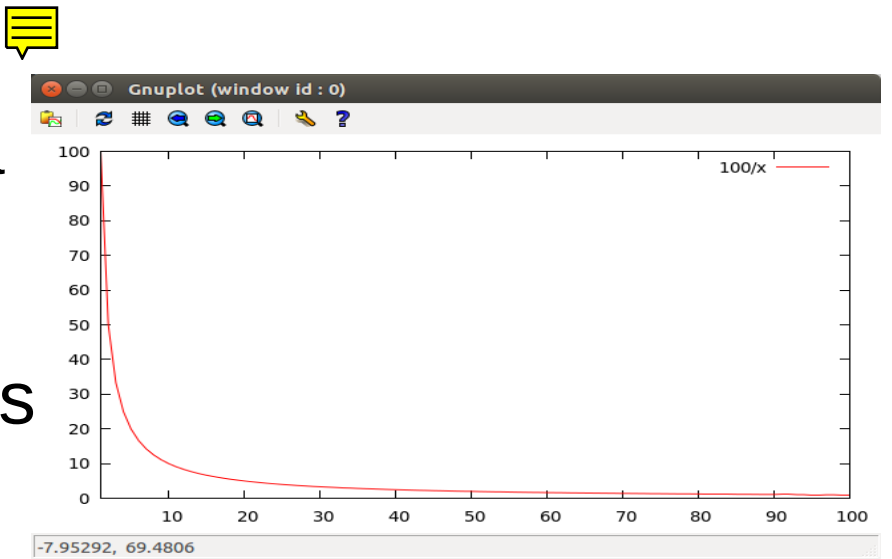
- Se basa en el hecho de un término que aparece en pocos documentos de la colección es muy específico y tiene un alto poder discriminador [cite: 1]. Términos comunes a casi todos los documentos aportan poca información distintiva

4. Ponderación de términos

- Un idf simple:
  Un término t_i aparece en n_i documentos de los N de la colección. Entonces:

$$idf_i = N/ n_i$$

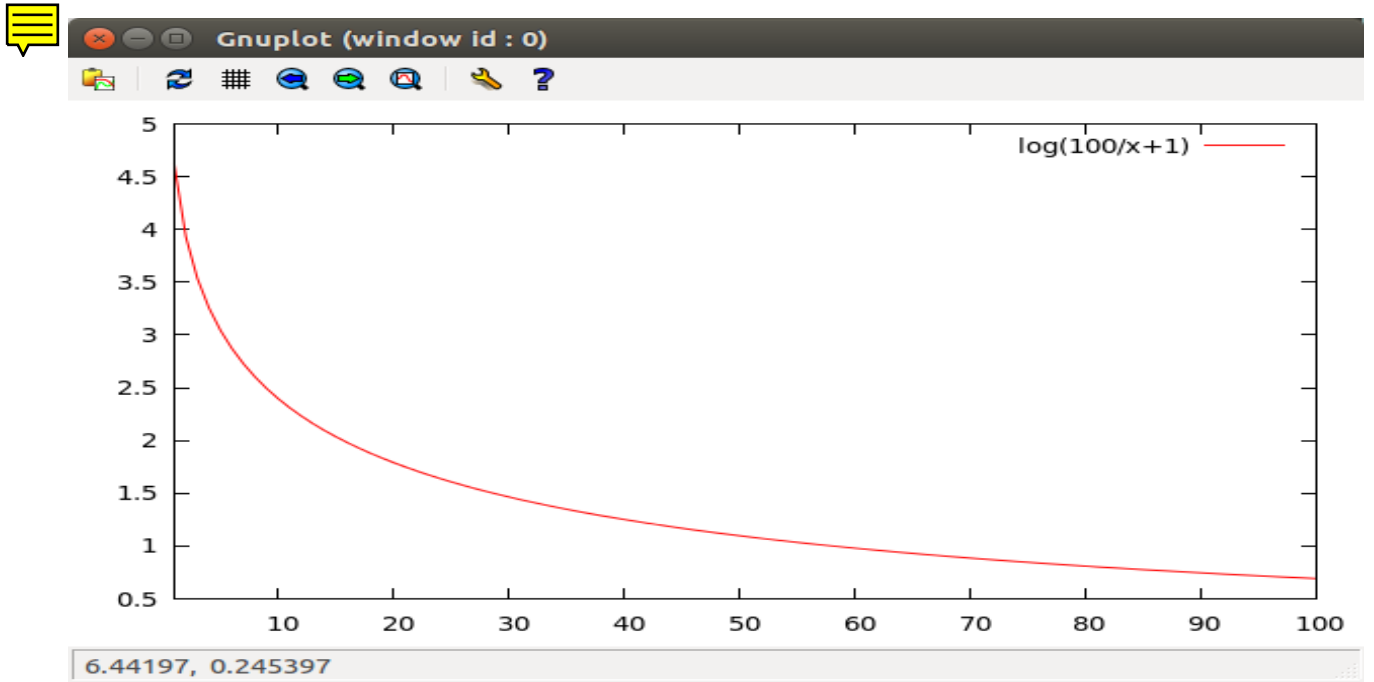
  Se daría un peso mayor a los términos que aparecen en menos documentos y un peso bajo a aquellos que aparecen en muchos.



Problema de este peso?

4. Ponderación de términos

- Tomando logaritmos para suavizar las diferencias...
  - $idf_i = log\ (N/ n_i + 1),\ n_i>0$



Variantes

Variants of IDF weight

weighting scheme	IDF weight ($n_t = \|\{d \in D : t \in d\}\|$)
unary	1
inverse document frequency	$\log \frac{N}{n_t}$
inverse document frequency smooth	$\log(1 + \frac{N}{n_t})$
inverse document frequency max	$\log\left(1 + \frac{\max_{\{t' \in d\}} n_{t'}}{n_t}\right)$
probabilistic inverse document frequency	$\log \frac{N - n_t}{n_t}$

Fuente: Wikipedia

4. Ponderación de términos

- Ejemplo: N = 1.000 documentos.

term i	n_i	N/n_i	idf_i
A	100	10,00	4,32
B	500	2,00	2,00
C	900	1,11	1,13
D	1.000	1,00	1,00

4. Ponderación de términos

- Pesos de los términos:
  - Combinación de tf e idf (ocurrencias en el documento y en la colección):

$$w_{ij} = tf_{ij} * idf_j = f_{ij} * (\log_2 (N/n_j) + 1),\ n_j > 0$$

  - Objetivo: asignar pesos más altos a aquellos términos que:
    - sean frecuentes en documentos relevantes, pero...
    - ... infrecuentes en la colección.

	...	amor	atentados	carrera	cuenta	de	espías	fuego	judio	la	munich	novela	olimpiadas	película	que	relato	...
A	...	0	1	0	1	5	1	0	0	2	2	2	1	1	5	2	...

si la colección tiene 1000 docs y los términos de, novela, espías y munich aparecen, respectivamente, en 1000, 700, 10 y 1 docs distintos

	...	amor	atentados	carrera	cuenta	de	espías	fuego	judio	la	munich	novela	olimpiadas	...
A	...	0	7.4	0	0.48	0	6.64	0	0	0	19.94	1.02	7.15	...

4. Ponderación de términos

- Pesos tf x idf normalizados:

  Solucionan el hecho de que en documentos más largos se asignen pesos más altos.

  W_{ij} peso del término i en el documento j

$$w_{ij} = \frac{tf_{ij}\, idf_i}{\sqrt{\sum_{k \in d_j} (tf_{kj})^2 (idf_k)^2}}$$

  w_{ij} ϵ [0, 1] resolviendo el problema de que documentos grandes reciban puntuaciones artificialmente altas por el simple volumen de ocurrencias.

Ejemplo

- Considerar una colección compuesta por los siguientes 5 documentos:

- D1= “Dogs eat the same things that cats eat”
- D2 = “No dog is a mouse”
- D3 = “Mice eat little things”
- D4 = “Cats often play with rats and mice”
- D5 = “Cats often play, but not with other cats”

Ejemplo (idf) -Cont

$df_{cat}=3$

$idf_{cat}=\ln(5/3)=0.51$

$df_{dog}=2$

$idf_{dog}=\ln(5/2)=0.91$

$df_{eat}=2$

$idf_{eat}=\ln(5/2)=0.91$

$df_{mouse}=3$

$idf_{mouse}=\ln(5/3)=0.51$

$df_{play}=2$

$idf_{play}=\ln(5/2)=0.91$

$df_{rat}=1$

$idf_{rat}=\ln(5/1)=1.61$

Ejemplo-Cont

- V1(cat, eat,dog)
  - $w_{cat} = tf_{cat} * idf_{cat} = 1 * 0.51 = 0.51$
  - $w_{dog} = tf_{dog} * idf_{dog} = 1 * 0.91 = 0.91$
  - $w_{eat} = tf_{eat} * idf_{at} = 2 * 0.91 = 1.82$
- V2(dog,mouse)
  - $w_{dog} = tf_{dog} * idf_{dog} = 1 * 0.91 = 0.91$
  - $w_{mouse} = tf_{mouse} * idf_{mouse} = 1 * 0.51 = 0.51$

Ejemplo-Cont

- V3(mouse,eat)
  - $w_{mouse} = tf_{mouse} * idf_{mouse} = 1 * 0.51 = 0.51$
  - $w_{eat} = tf_{eat} * idf_{at} = 1 * 0.91 = 0.91$
- V4(cat,mouse,play, rat)
  - $w_{cat} = tf_{cat} * idf_{cat} = 1 * 0.51 = 0.51$
  - $w_{play} = tf_{play} * idf_{play} = 1 * 0.91 = 0.91$
  - $w_{rat} = tf_{rat} * idf_{rat} = 1 * 1.61 = 1.61$
  - $w_{mouse} = tf_{mouse} * idf_{mouse} = 1 * 0.51 = 0.51$

Ejemplo-Cont

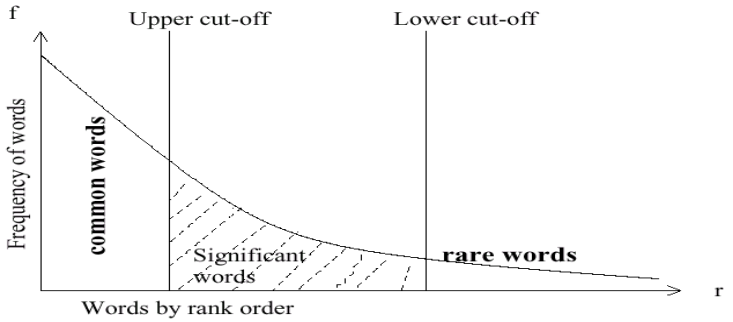
- V5
  - $w_{cat} = tf_{cat} * idf_{cat} = 2 * 0.51 = 1.02$
  - $w_{play} = tf_{play} * idf_{play} = 1 * 0.91 = 0.91$

5. Reducción de vocabulario y umbrales de frecuencia

- En algunos casos, no es suficiente filtrar las palabras vacías de significado y aplicar segmentación / lematización.
- Se necesita reducir más el vocabulario, e identificar los mejores términos para la recuperación.
- Se aplica un proceso de *Selección de términos*.

5. Reducción de vocabulario y umbrales de frecuencia

Métodos basados en la frecuencia de aparición:



Palabras importantes (según Luhn):

- Aquellas que son capaces de discriminar el contenido de documentos se sitúan en medio de dos umbrales.
  - Umbral Superior (Upper cut-off) : Elimina palabras extremadamente comunes y genéricas sin poder discriminativo
  - Umbral Inferior (Lower cut-off) : Elimina palabras extremadamente raras o errores tipográficos que no aportan solidez estadística

Representación vectorial de término

- ¿Cómo podemos conocer la semántica de un término?
- Estará asociada a un conjunto de características
  - perro: ser vivo, forma ovalada, de tamaño pequeño, que tiene pelo, patas, ojos y ladra
  - moto: tamaño pequeño, de metal, con ruedas
- Podemos imaginar cada una de estas características como una dimensión de un vector donde los valores representan representando la importancia de la característica a la hora de describir el concepto.

	vivo	ovalado	grande	pelo	negro	marrón	patas	ruedas	ladra	maulla	metal	ventana	ojos	faros
perro	1	0.7	0.3	1	0	0.5	1	0	1	0	0	0	1	0
coche	0.	0.8	0.9	0	0.6	0.2	0	1	0	0	1	1	0	1
gato	1	0.8	0.1	1	0.3	0.6	1	0	0	1	0	0	1	0
bici	0	0.2	0.5	0	0.3	0.3	0	1	0	0	1	0	0	1

Representación vectorial de términos

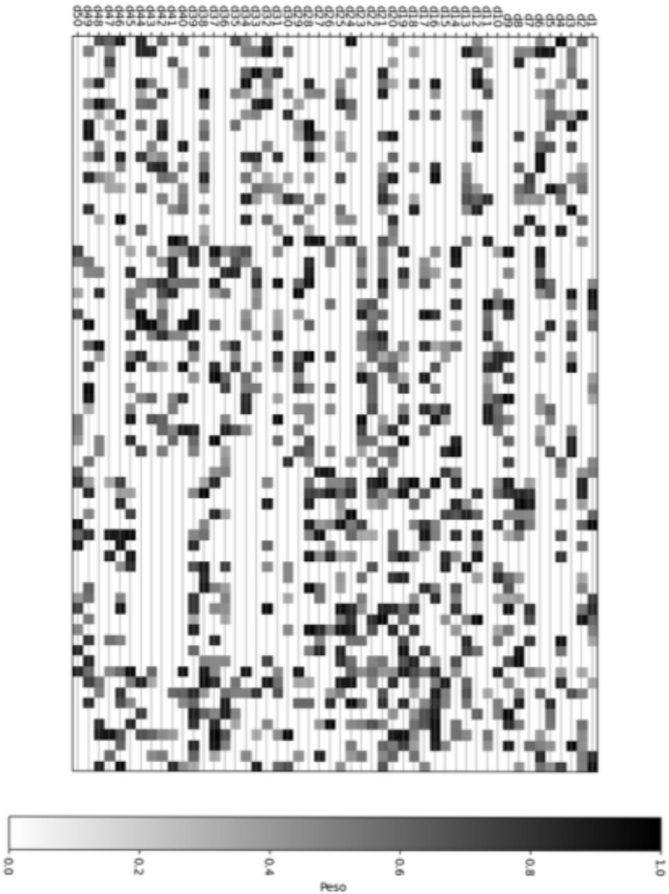
- ¿cómo podemos extraer la semántica asociada a una palabra a partir únicamente del texto escrito?
  - Utilizaremos el contexto: palabras que aparecen en contextos similares suelen tener significados similares.
    - perro y gato aparecen en documentos relacionados con mascotas,
    - moto y coche son comunes en documentos relacionados con la mecánica o transportes.
    - todas puede aparecer juntas en docs relacionados con el juego.

Representación vectorial de términos

- presentamos el significado de una palabra por los documentos que la contienen.
  - tenemos tantas dimensiones como documentos
  - en cada dimensión almacenamos la importancia de la palabra en el documento (medida con TF o TF-IDF).

problemas:

el número de dimensiones, del orden de millones y la repres. varía entre colecciones o cuando cambia el tamaño de la colección.



Representación vectorial de términos

- Otra alternativa: *utilizar exclusivamente las relaciones entre términos*.
  - El vector asociado a un término tendrá tantas dimensiones como términos existan en el vocabulario,
  - El valor asociado a cada dimensión dependerá del número de veces que los dos términos aparezcan simultáneamente en un contexto (frase, párrafo, N palabras

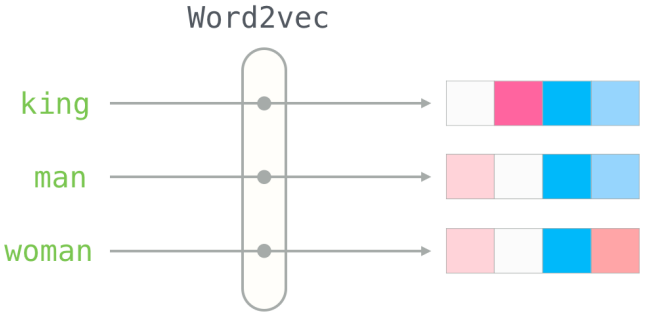
El coche se averió en el día en que pensábamos viajar. Si consideramos como contexto una frase, coche y viajar sí estarían relacionados

- Si consideramos una ventana de 5 no no lo estarían

	jugar	mascota	animal	mecánico	comer	correr	viajar	...
...				...				
perro	0.6	0.7	0.7	0	0.6	0.2	0.01	...
gato	0.4	0.8	0.6	0.01	0.4	0.1	0.01	...
coche	0.5	0.05	0	0.8	0	0.5	0.9	...
bici	0.6	0	0	0.5	0	0.8	0.6	...
...				...				

Desafíos de los enfoques tradicionales: Representaciones de muy alta dimensión, 'sparse' (llenas de ceros) e incapaces de capturar relaciones sutiles o similitudes entre palabras no vistas juntas en la misma ventana exact

Word2Vec



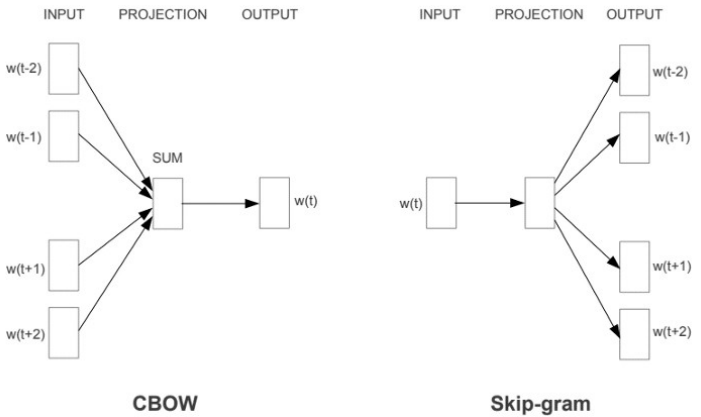
- Word2Vect es una técnica para poder aprender estos vectores (embeddings) desarrollada por Tomas Mikolov y su equipo en Google en 2013.

  Utiliza una red neuronal para predecir si una palabra determinada, como perro, es probable que aparezca cerca de otra, como mascota.

  - Si en numerosos textos estas palabras suelen encontrarse en contextos similares, el modelo refuerza los pesos que han llevado a dicha predicción, ajustándolos progresivamente a medida que se procesan más datos.
- Ventajas:
  - Reduce las dimensiones 100 – 430, No tienen una interpretación clara.
  - Son vectores densos.
  - Permite alcanzar un mejor rendimiento en distintas tareas de PLN como búsqueda semántica, traducción automática y análisis de sentimientos.

Word2Vec

- Modelos de redes neuronales: Se utilizan dos arquitecturas principales:
  - CBOW (Continuous Bag of Words): Predice una palabra central basándose en sus palabras de contexto inmediatas.
  - Skip-gram: Predice las palabras del contexto basándose en una palabra central.



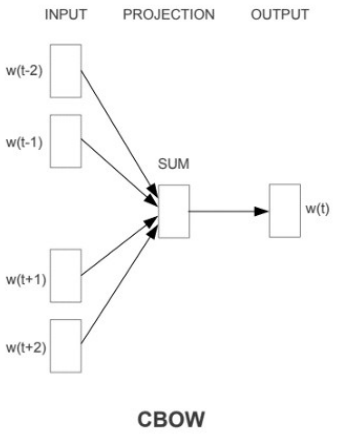
Word2Vec

- Modelos de redes neuronales: Se utilizan dos arquitecturas principales:
  - CBOW (Continuous Bag of Words): Predice una palabra central basándose en sus palabras de contexto inmediatas (ventana).

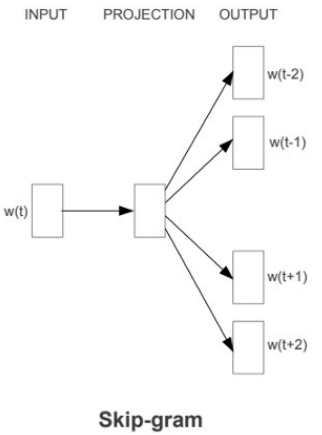
    El coche se averió en el xxxx en que pensábamos viajar.

    - 2 palabras para definir el contexto en el xxxx en que
      - es mas probalbe términos como lugar, momento, instante, caso
      - poca probabilidad a juego, perro, comer.
    - 4 palabras: se averió en el xxxx en que pensábamos viajar
    - + probable momento, instante, día - prob. caso lugar
    -
  - Skip-gram:

    Predice las palabras del contexto basándose en una palabra central.



CBOW

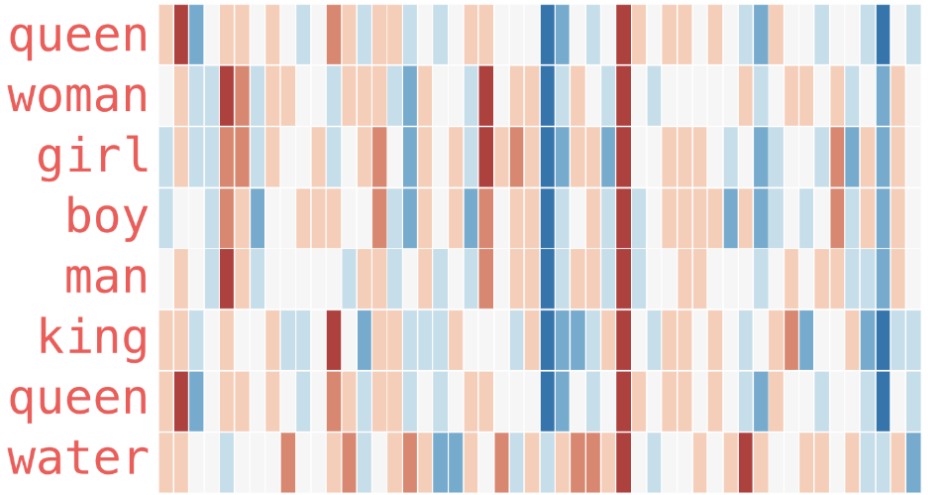


Skip-gram

Word2Vec

- Los embeddings (vectores) aprendidos son capaces de capturar el contexto, la similitud semántica y sintáctica (género, sinónimos, etc.) de una palabra.

KING [0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 , -0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 , -0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 , -1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042]



Word2Vec: Aritmética Vectorial y Analogías:

- Se pueden realizar operaciones algebraicas (suma y resta) sobre vectores y tienen sentido

  Madrid - España + Francia = París

  Podemos utilizar estos vectores para la detección de:

  - analogías, como en: Rey - Hombre + Mujer = Reina
  - de número, como en: gato - gatos + perros = perro

- Las relaciones aprendidas con Word2Vec pueden ser peligrosas porque reflejan los sesgos presentes en los datos de entrenamiento: puede absorber estereotipos, prejuicios y asociaciones problemáticas.
  - Por ejemplo, si consideramos las obras de Blasco Ibáñez, los términos mas similares a campo son cortijo, **inculto**, jornalero, solo podemos decir que aparecen de forma cercana en los datos
- Sesgos que se pueden aprender de género o el racial, como en:
  - médico - hombre + mujer = enfermera
  - violencia + musulman = terrorismo

Glove: Embeddings basados en conteos globales

Creadores: Jeffrey Pennington, Richard Socher y Christopher D. Manning (Stanford University, 2014).

Inspiración: Combina las ventajas de las dos familias principales de algoritmos:

- Métodos Matriciales Globales (LSA, LSI): Aprovechan estadísticas globales del corpus pero no funcionan bien en analogías.
- Métodos de Ventana Predictiva Local (Word2Vec): Capturan estructuras complejas de analogía pero ignoran la información estadística global del corpus.

Idea Clave: Se entrena sobre la matriz global de coocurrencia término-término X, donde X_ij es el número de veces que el término i aparece en el contexto del término j.

Glove: Embeddings basados en conteos globales

Razón de Probabilidades Condicionales (El núcleo de GloVe)

- GloVe la información semántica fundamental entre dos palabras w_i y w_j se extrae observando la razón de sus probabilidades condicionales con respecto a una palabra sonda k: P(k | w_i=hielo) / P(k| w_j=vapor)
-

Palabra clave (k)	¿Aparece cerca de **"hielo"**?	¿Aparece cerca de **"vapor"**?
Sólido	**Muchas veces**	Muy pocas
Gas	Muy pocas	**Muchas veces**
Agua	**Muchas veces**	**Muchas veces**
Moda	Casi nunca	Casi nunca

Glove: Embeddings basados en conteos globales

Razón de Probabilidades Condicionales (El núcleo de GloVe)

- 1. **Para "Sólido":**
     - $\frac{\text{Alta}}{\text{Baja}} = \textbf{Número muy GRANDE } (> 1)$
     - *Conclusión:* "Sólido" está muy relacionado con el **hielo**, pero no con el vapor.
  2. **Para "Gas":**
     - $\frac{\text{Baja}}{\text{Alta}} = \textbf{Número muy PEQUEÑO } (< 1)$
     - *Conclusión:* "Gas" está muy relacionado con el **vapor**, pero no con el hielo.
  3. **Para "Agua":**
     - $\frac{\text{Alta}}{\text{Alta}} = \textbf{Cercano a 1}$
     - *Conclusión:* "Agua" es una propiedad **común** que comparten ambas palabras.
  4. **Para "Moda":**
     - $\frac{\text{Baja}}{\text{Baja}} = \textbf{Cercano a 1}$
     - *Conclusión:* "Moda" **no tiene relación** con ninguna de las dos (es ruido irrelevante).

En GloVe no se seleccionan manualmente palabras clave. Las palabras clave del ejemplo ("sólido", "gas", "agua") eran solo una simplificación para entender la idea.

Glove: Embeddings basados en conteos globales

Generación de Vectores:

1.- **Construcción del Vocabulario:** Se toma todo el texto del corpus y se extraen todas las palabras únicas que existen en él. Esto genera un vocabulario V de, por ejemplo, 400.000 palabras distintas.

1.1.- **Matriz Gigante de Coocurrencia**: GloVe crea una matriz cuadrada gigante de tamaño VxV (400.000 x 400.000). Todas las palabras son filas y también columnas.

Cada celda X_{ij} de la matriz almacena cuántas veces la palabra j apareció dentro de una "ventana" de visión alrededor de la palabra i.

2. Generar los vectores

Transformar X en vectores numéricos pequeños y densos (por ejemplo, de 100 o 300 dimensiones) es un proceso de optimización matemática.

multiplicar el vector de la palabra w_j x w_j sea aprox igual a $\log(P(w_i|w_j))$

GloVe: estructura de ficheros

Es un archivo de texto plano (.txt): No utiliza un formato binario ni complejo. Está separado simplemente por espacios en blanco.

- clave: El primer elemento de cada línea es la palabra misma (en minúsculas).
- vector: Cada número flotante a continuación representa una dimensión del espacio vectorial.
  - Si descargas la versión de 50 dimensiones (50d), cada línea tendrá 51 elementos (1 palabra + 50 números).
  - Si descargas la versión de 300 dimensiones (300d), cada línea tendrá 301 elementos.

Orden de frecuencia: suele estar ordenado por la frecuencia con la que aparecen las palabras en el texto de entrenamiento.

```
the 0.418 0.24968 -0.41242 0.1217 0.34527 -0.044457 -0.49688 -0.17862 -0.0006
of 0.013441 0.23682 -0.16899 0.40951 0.63812 0.027273 -0.29952 -0.0033013 0.0
one -0.39922 0.05923 0.32049 0.13885 0.22271 -0.18341 -0.44973 0.018596 0.123
in 0.10008 0.29938 -0.42878 0.30133 0.66411 -0.012351 -0.34354 0.13452 -0.076
cat 0.21312 -0.09823 0.11234 -0.54321 0.08912 0.32145 -0.12345 0.45678 -0.012
king 0.50451 0.68607 -0.59517 -0.022801 0.60047 -0.13498 -0.08813 0.47377 -0.
queen 0.35412 0.51234 -0.41123 0.01234 0.58912 -0.09823 -0.01234 0.38912 0.02
```